



FinTraceQA

A Multilingual Multi-Hop Question Answering System for Indian Financial Markets

22AIE315 - Natural Language Processing Team 4

Course: Natural Language Processing — Semester 6 | **Domain:** Financial NLP, Knowledge Graphs, Neural Machine Translation | **Date:** March 2026

Building a Reasoning Machine for Indian Finance

FinTraceQA is an end-to-end question answering system designed specifically for the Indian financial domain. Unlike conventional QA tools that retrieve a single fact from a document, FinTraceQA is built to reason across multiple connected pieces of information stored in a structured Knowledge Graph.

A user can ask, in any of seven languages, a complex question like *"Which NIFTY-50 company was most impacted by the RBI rate cut announced in early 2024?"* and the system will traverse a chain of graph relationships, score the best reasoning path, and return a grounded, explainable answer in the same language the question was asked.

Financial Knowledge Graph

33,300 entities and 10,000 relationships from NIFTY-50 data, IN-FINews corpus, macro-economic indicators, stored in Neo4j Aura cloud

Multi-Hop QA Backend

FastAPI system that detects language, translates to English, decomposes questions, retrieves subgraphs, scores paths, synthesizes answers

Finterest Mobile App

React Native interface with chat screen, live news feed, explanation panel, multilingual support

Why Existing Systems Are Not Enough

Single-Hop Limitation

Traditional systems only fetch single facts. Financial reasoning requires chaining (e.g., finding entities linked to Mukesh Ambani → filtering by oil sector → measuring post-2020 price price drops).

Language Barrier

Over 55% of India's internet users prefer regional languages for finance, yet existing tools operate exclusively in English, locking out millions of retail investors.

LLM Hallucination

Large language models produce fluent-sounding answers wit

The gap that FinTraceQA fills: **multi-step structured reasoning + multilingual query understanding + transparent answer provenance**. No prior system addresses all three simultaneously for the Indian financial domain.

Objectives and Research Questions

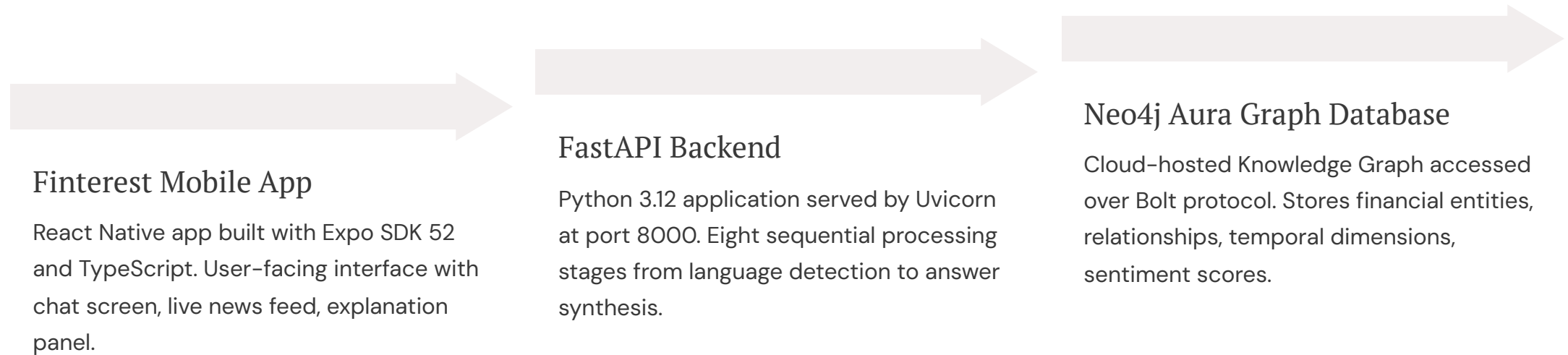
Project Objectives

01	02	03
Domain-Specific Financial KG Build a Knowledge Graph for Indian finance, capturing entities, relations, context, and provenance.	Multi-Hop Reasoning Engine Implement a multi-hop reasoning engine for complex financial queries.	Custom Multilingual NMT Train a custom multilingual NMT for English and six major Indian languages.
04	05	
Explainable Reasoning Pipeline Assemble components into an explainable reasoning pipeline with transparent chains.	 Usable Mobile Application Deploy FinTraceQA as an accessible mobile application.	

Guiding Research Questions

KG vs. LLM Reasoning Can a structured Knowledge Graph surpass LLM retrieval for multi-hop financial reasoning accuracy?	NMT Translation Quality Is a 176M-parameter Transformer sufficient for exact entity preservation in financial translation?	Prompt-Based Extraction Reliability Can prompt-based extraction with Gemini 2.5 Flash reliably extract financial entities and relations?
---	--	--

System Architecture: Three-Layer Design



When a query arrives, it passes through eight stages: LanguageRouter detects script and language using Unicode block analysis, LocalTranslator converts non-English queries to English, QuestionComposer classifies question type and generates traversal plan, EntityLinker and RelationLinker resolve named entities and predicates, KGTraversalEngine executes Cypher queries and scores paths, AnswerSynthesizer passes top-scoring triples to Gemini for natural language generation, FaithfulnessVerifier checks answer support, and final translation returns answer in original language.

Knowledge Graph Construction: What is a KG?

A Knowledge Graph (KG) is a powerful way to organize information, representing data as interconnected entities and relationships. It forms the backbone of FinTraceQA's reasoning capabilities.

KG Definition

A KG is a directed labeled multigraph $G = (V, E, L)$ where V are entity nodes, $E \subseteq V \times V$ are directed edges, and $L : E \rightarrow R$ assigns relation types. An atomic fact is a triple (s, r, o) , e.g., "**Infosys OPERATES_IN IT Sector**".

Text Pre-processing

Raw text must be meticulously cleaned before extraction:

- **Sentence Boundary Detection:** Splits text into sentences, handling complex cases like "Rs. 2,450".
- **Tokenization:** Breaks sentences into words or subwords. For Indic scripts like Tamil, SentencePiece Unigram learns segmentation from character frequencies.
- **Normalization:** Lowercasing, Unicode NFKC normalization, abbreviation expansion, and noise removal ensure data consistency.

Named Entity Recognition (NER)

NER identifies and classifies entity spans in text (e.g., "Infosys" as a **Company**, "RBI" as an **Organization**). It assigns labels using the BIO tagging scheme ($y_i \in \{\text{B-TYPE, I-TYPE, O}\}$).

Methods:

- **Rule-based:** Fast, interpretable, but lacks generalization.
- **Classical ML (CRF):**
$$P(y|w) = \frac{1}{Z(w)} \exp\left(\sum_{t=1}^n \sum_k \lambda_k f_k(y_{t-1}, y_t, w, t)\right)$$

Knowledge Graph Construction: Key Processes

Building a robust Knowledge Graph involves several critical steps after Named Entity Recognition, focusing on how entities relate, disambiguating mentions, and resolving ambiguities.

Relation Extraction (RE)

RE identifies relationships between recognized entities. For an entity pair (e_i, e_j) , a classifier predicts the relation $r \in R \cup \{\text{None}\}$. Dependency-path RE extracts the shortest syntactic path between entities (e.g., "Infosys **reported** profit" suggests a REPORTS relation).

Coreference Resolution

This process links different expressions in a text that refer to the same real-world entity, creating coreference chains. For example, linking "The company" to "Reliance Industries" and "it" to "the refinery project". Without this, entities would be duplicated, leading to an incomplete KG. Mention Detection finds candidate noun phrases by scoring spans.

Entity Linking & Disambiguation (EL/ELD)

- EL/ELD maps a text mention (e.g., "Wipro") to its unique canonical node in the Knowledge Graph, resolving ambiguities. It generates candidate entities from the KG and ranks them based on context-entity compatibility.
$$\hat{e} = \operatorname{argmax}_{e \in C(m)} \operatorname{score}(m, e, \text{context}).$$
- This formula represents how the system selects the best entity candidate from the set of possible candidates based on a scoring function that considers the mention, entity, and context.

Knowledge Graph Construction



Entity Deduplication

After extraction from multiple documents, the same real-world entity often appears under many surface forms. For example, "Tata Consultancy Services", "TCS", "Tata Consulting", and "TCS Ltd" all refer to the same node. Canonicalization merges these disparate mentions into one unique node with a stable identifier, preventing duplication and ensuring a coherent Knowledge Graph.



String-Based Deduplication

This approach utilizes the SequenceMatcher ratio, calculated as $sim(a, b) = 2M / (|a| + |b|)$, where M is the number of matching characters and $|a|$, $|b|$ are the lengths of the strings. It is fast and requires no complex models, performing well for highly similar strings. In FinTraceQA, a threshold of 0.85 was used. However, it fails to link abbreviations to full names (e.g., "TCS" vs. "Tata Consultancy Services" has a similarity score close to 0.0).



Embedding-Based Deduplication

This method involves encoding entity names and descriptions using a sentence encoder and clustering them based on cosine similarity, expressed as $\cos\theta = \hat{u} \cdot \hat{v}$. Dense retrieval with FAISS enables $O(\log n)$ approximate nearest-neighbor searches. While powerful for semantic similarity, it demonstrated catastrophic failure in FinTraceQA for short financial names when using models like all-MiniLM-L6-v2, indicating limitations in specific domains.



Cross-Batch Deduplication

The IncrementalKGBuilder runs Union-Find clustering using SequenceMatcher similarity $sim(a, b) = 2M / (|a| + |b|) \geq 0.85$ over entity pairs with the same label. A noise filter prevents the merging of pure numbers, currency fragments, and lowercase short strings using case-sensitive regex, explicitly preserving crucial uppercase acronyms like TCS, NSE, BSE, and RBI. Stable identifiers are generated using the formula

$id(n, l) = slug(n)[: 32] || SHA256(lower(n) :: lower(l))[: 8]$.

Gemini Atomic Fact Extraction

Turning Financial Text Into Structured Graph Triples

The `GeminiAtomicExtractor` class sends each text chunk to Gemini 2.5 Flash with a structured prompt, requesting a JSON array of atomic facts. Each fact precisely records a subject entity (with type), a predicate relation, an object entity (with type), a confidence score, a sentiment score, and four distinct temporal timestamps.

Confidence & Sentiment

Each fact includes a confidence score $c \in [0, 1]$ and a sentiment score $s \in [-1, +1]$, reflecting whether the relationship is **bullish** or **bearish** for the involved entities.

Entity & Relation Types

Supports six entity types (Company, Person, Sector, Event, Policy, Indicator) and fourteen relationship types including REPORTS, ANNOUNCES, IMPACTS, and LEADS.

Four Temporal Dimensions

Announce Time t_{ann}

When the information was publicly disclosed.

Effective Time t_{eff}

When the described relationship took legal or business effect.

Observation Time t_{obs}

When the data was ingested into the Knowledge Graph.

Impact Time t_{imp}

When the observable market effect was measured (e.g., price movement).

This semantic distinction in temporal timestamps is crucial for accurate temporal queries, ensuring user intent (e.g., asking about RBI rate cuts) is matched to the correct time dimension, not just data ingestion time. After extraction, the facts are mapped into the storage format for the Incremental KG Builder.

Multi-Hop Question Decomposition



Coreference Resolution

Replaces pronouns with session context entities



Question Classification

Four types identified by patterns



Traversal Plan

Ordered sub-questions generated



Single-Hop

One graph lookup required. Patterns: "what is", "who is", "ceo of", "sector of"



Multi-Hop Chain

Follow chain of edges. Patterns: "as a result of", "impact of X on Y", "how does X affect Y"



Multi-Hop Intersection

Find intersection of entity sets. Patterns: "both ... and", "common between"



Temporal

Add time filter to any type. Identified by temporal prepositions near date references

The QuestionDecomposer performs coreference resolution against session memory, classifies question type, and generates a DecompositionPlan with ordered SubQuestion objects. The primary path sends classified question to Gemini requesting JSON with question_type, subquestions, anchor_entities, and constraints. Fallback uses deterministic regex pattern matching against 30+ hand-crafted patterns per type.

Entity and Relation Linking

Entity Linking Cascade

Three-tier cascade with scored candidates:

- 1. **Exact substring matching:** Case-insensitive substring predicate, score $S_1=1.0$
- 2. **Alias lookup:** Curated financial dictionary maps abbreviations to canonical forms, score $S_2=0.85$
- 3. **Fuzzy matching:** difflib.SequenceMatcher with ratio ≥ 0.55 , score $S_3=0.65 \times \text{ratio}$

Cascade stops at first tier returning results. All candidates deduplicated by node ID, sorted by score descending, top 5 returned.

01

Extract n-gram spans from query
(length 4 to 1)

02

Filter spans containing only
stopwords

03

Apply three-tier cascade matching

04

Score and rank candidates

05

Return top 5 with highest scores

Relation Linking

Hand-crafted predicate map associates ~60 surface patterns with KG relation types:

- **REPORTS:** "quarterly", "earnings", "results", "filing"
- **ANNOUNCES:** "announced", "launched", "merger", "IPO"
- **IMPACTS:** "impacted", "affected", "fell", "dropped"
- **OPERATES_IN:** "sector", "industry", "business"

Patterns sorted by descending length so multi-word patterns override shorter overlaps.

FinTraceQA's Novel Extraction Design

FinTraceQA employs a multi-stage, mathematically-grounded approach for robust financial information extraction, prioritizing precision and reproducibility over traditional embedding-based methods.

1. Joint LLM Atomic Fact Extraction

One structured Gemini 2.5-Flash prompt extracts atomic facts, each an indivisible triple, with fixed temperature $T = 0.0$ for deterministic output.

$$f_i = \langle s_i, \tau_s, p_i, o_i, \tau_o, c_i, \sigma_i, \mathbf{t}_i, \text{src}_i \rangle$$

2. Four-Dimensional Temporal Representation

Each fact includes four distinct timestamps: announcement (t_{ann}), effective (t_{eff}), observation (t_{obs}), and impact (t_{imp}) time.

$$\mathbf{t}_i = (t_{\text{ann}}, t_{\text{eff}}, t_{\text{obs}}, t_{\text{imp}})$$

3. Parallel Async Batching

Dispatches all text sections simultaneously via `asyncio.gather`, reducing wall-clock time from $O(N)$ to $O(1)$ and achieving ~10x speedup.

4. Intra-Call Entity Cache Deduplication

Reuses entity objects within a single Gemini call via a normalized cache key, preventing duplicate nodes before global deduplication.

$$k = \text{lower}(\tau) \parallel \text{"::"} \parallel \text{lower}(\text{name})$$

5. Stable Deterministic Node ID

Assigns collision-resistant IDs using a slug and SHA-256 hash suffix for uniqueness.

$$\text{id}(n, \ell) = \text{slug}(n)_{[:32]} \parallel \text{"_"} \parallel \text{SHA-256}(\text{lower}(n) \parallel \text{"::"} \parallel \text{lower}(\ell))_{[:8]}$$

6. Noise Filtering Before Dedup

Blocks pure numbers, currency fragments, generic phrases, and short lowercase strings (e.g., "buy"), while preserving acronyms like TCS, RBI due to case-sensitive regex.

7. Union-Find String-Similarity Deduplication

Uses disjoint-set union-find with a two-strategy merge predicate (SequenceMatcher ratio $\geq \theta$ or suffix-stripped prefix match), optimized with a prefix-index for $O(K \times N_{\text{new}})$ complexity.

$$r(a, b) = \frac{2M}{|a| + |b|}$$

8. Canonical Entity Selection

Chooses representative by a multi-factor score preferring longer proper names, balancing Gemini confidence and word count.

$$\text{score}(e) = c_e \times \min\left(\frac{w_e}{2}, 1.5\right) \times \begin{cases} 0.5 & |n_e| \leq 2 \\ 1.0 & \text{otherwise} \end{cases}$$

9. Attribute Merging

Combines attributes by taking the maximum confidence and filling gaps with incoming values without overwriting.

Why Embeddings Were Abandoned

The system deliberately **regresses from neural to lexical similarity** for deduplication. Embedding-based clustering led to catastrophic over-merging of short financial acronyms (e.g., TCS, NSE, RBI) due to their high cosine similarity, despite being distinct entities. String similarity (e.g., $r(\text{"TCS"}, \text{"NSE"}) = 0.0$) correctly identifies them as unique, proving more reliable for this domain's unique characteristics.

Session Memory & Multi-Turn Conversations

To maintain conversational context, FinTrace employs a robust session memory system, enabling seamless multi-turn interactions.

SessionMemory Class

Manages a per-session sliding window of conversation history.

- `collections.deque` with max length 10
- Stores query, answer, entities, KG node IDs, and event names
- Thread-safe operations via `threading.Lock`

`get_context()` Method

Aggregates entity references for contextual understanding.

- Considers most recent $\min(|\text{history}|, e_{\text{ttl}})$ turns ($e_{\text{ttl}}=5$)
- Returns `e_last`, `e_last_id`, `e_event`, and $\mathcal{E}_{\text{recent}}$
- Enables pronoun substitution in `QuestionDecomposer`

The global `SessionStore` maps UUID session IDs to `SessionMemory` instances, created on demand and protected by a lock.

📌 For production scale, future work includes implementing a bounded LRU dictionary or external Redis store for TTL-based session eviction.

Translation Model

Encoder · Decoder · Tokenizer · Mathematics · Loss · Decoding Functions

SentencePiece Tokenizer — Unigram Model

What It Is

SentencePiece Unigram model trained from scratch on multilingual Indian language corpus. Starts with large candidate vocabulary, iteratively prunes by removing subwords that minimize total loss, converging to $V = 100,000$ subword units.

Training Data

AI4Bharat Samanantar corpus — 20,000 samples per language from Hindi, Tamil, Telugu, Marathi, Kannada, Malayalam plus English. Both source and target text written to same corpus file for unified multilingual vocabulary.

Unigram Objective

The probability of the most likely segmentation is given by the formula:

$$P(X) = \Sigma P(x)$$

where the probability of a subword sequence x is a product of individual subword probabilities:

$$P(x) = \Pi p(x_i)$$

Vocabulary pruning is achieved by maximizing the log-likelihood:

$$L = \Sigma \log P(X^{(s)})$$

Key Configuration

vocab_size=100K, model_type=unigram,
character_coverage=0.9995, split_by_unicode_script=True,
byte_fallback=False

Special Tokens

[PAD], [UNK], [CLS]/BOS, [SEP]/EOS, [MASK], plus language direction tokens <2hi>, <2ta>, <2te>, <2mr>, <2kn>, <2ml>

Fertility Metric

Measures tokens per word. Target ≤ 2.5 tokens/word across all Indic languages.

Encoder Architecture



Input Preparation

$$emb(x_i) = E[x_i] \cdot \sqrt{d_{model}} \approx 27.7$$

Raw source token IDs are passed through

a shared embedding matrix $E \in R^{(V \times d)}$

where $V = 100,000$ and $d = 768$.

Embeddings are scaled as shown above.



Sinusoidal Positional Encoding

$$PE(pos, 2i) = \sin(pos/10000^{(2i/d)})$$

$$PE(pos, 2i + 1) = \cos(pos/10000^{(2i/d)})$$

Positional encodings use sine and cosine functions with geometrically increasing frequencies, allowing the model to learn long-range and local positional information. Dropout $p = 0.1$ is applied after addition.



Pre-LayerNorm Architecture

$$SA_{out} = x + \text{Dropout}(\text{MHA}(\text{LN}(x), \text{LN}(x), \text{LN}(x), M_T))$$

$$\text{Enc}_{out} = SA_{out} + \text{Dropout}(\text{FFN}(\text{LN}(SA_{out})))$$

The encoder consists of 6 identical layers utilizing a Pre-LayerNorm architecture, where normalization occurs before the sublayer. This design with residual connections improves training stability and performance.



Multi-Head Attention

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k})V$$

Comprising $H = 12$ parallel heads, each with $d_k = 64$ dimensions, multi-head attention calculates scaled dot-product attention. A padding mask fills attention logits with $-\infty$ before softmax to ignore padded tokens.



Feed-Forward Network

$$\text{FFN}(x) = W_2 \cdot \text{Dropout}(\text{GELU}(W_1 x))$$

This is a two-layer MLP with GELU activation and $4\times$ expansion, where $W_1 \in R^{(768 \times 3072)}$ and $W_2 \in R^{(3072 \times 768)}$. GELU applies smooth probabilistic gating for better gradient flow.

Decoder Architecture



Decoder Seed

$$tgt_{(t=0)} = \text{ID}(< 2hi >)$$

The decoder is seeded with a target language token (e.g., `<2hi>`) to indicate the desired translation direction.



Causal Masking

$$M_{causal}[i, j] = \begin{cases} -\infty & \text{if } j > i, \\ 0 & \text{else} \end{cases}$$

A causal mask prevents the decoder from attending to future tokens during training, ensuring sequential processing.



Masked Causal Self-Attention

$$h^{(1)} = x + \text{Dropout}(\text{MHA}_{causal}(\text{LN}(x), \text{LN}(x)))$$

The first sublayer applies masked multi-head self-attention on the decoder's input, preventing information leakage from future tokens.



Cross-Attention over Encoder

$$h^{(2)} = h^{(1)} + \text{Dropout}(\text{MHA}_{cross}(\text{LN}(h^{(1)}), \text{enc}, \text{enc}))$$

This sublayer allows the decoder to attend to the encoder's output, bridging information between the source and target sequences.



Feed-Forward Network

$$h^{(3)} = h^{(2)} + \text{Dropout}(\text{FFN}(\text{LN}(h^{(2)})))$$

A standard two-layer feed-forward network processes the output from the cross-attention sublayer.



Output Projection with Weight Tying

$$\text{logits} = h^{(3)} W_{out} \in \mathbb{R}^{(B \times L_{tgt} \times V)} \quad \text{where} \quad W_{out} = W_E^T$$

The final output projects the decoder's representation to the vocabulary dimension. Weight tying with the embedding matrix reduces parameters and improves consistency.

Optimizer & Learning Rate Schedule

The Transformer model employs the AdamW optimizer with a specific learning rate schedule to effectively manage training dynamics and convergence.

AdamW Optimization

$$\begin{aligned}m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t \\v_t &= \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \\\hat{m}_t &= m_t / (1 - \beta_1^t) \\\hat{v}_t &= v_t / (1 - \beta_2^t) \\\theta_{t+1} &= \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \varepsilon} \hat{m}_t - \eta \lambda \theta_t\end{aligned}$$

These equations describe the calculation of momentum, RMSprop-like terms, their bias correction, and the final weight update including weight decay.

Key Hyperparameters

$$\begin{aligned}\beta_1 &= 0.9 \\\beta_2 &= 0.98 \\\varepsilon &= 10^{-9} \\\lambda &= 0.01 \\\eta_{peak} &= 5 \times 10^{-4} \\\text{max_grad_norm} &= 1.0\end{aligned}$$

The values for $\beta_2 = 0.98$ and $\varepsilon = 10^{-9}$ are adopted from Vaswani et al. (2017), contributing to a cautious approach during the warmup phase.

Warmup & Linear Decay Learning Rate Schedule

$$\eta(t) = \begin{cases} \eta_{peak} \cdot t / t_{warm} & \text{for } t \leq t_{warm} \\ \eta_{peak} \cdot (T_{total} - t) / (T_{total} - t_{warm}) & \text{for } t > t_{warm} \end{cases}$$

Where $t_{warm} = 4,000$ steps, T_{total} is the total training steps, and $\eta_{peak} = 5 \times 10^{-4}$. This matches the original Transformer's linear warmup and subsequent linear decay.

Greedy Decoding with Repetition Control

1

Temperature Scaling

$$\hat{l}_j = l_j / T, p_j = e^{\hat{l}_j} / \sum_k e^{\hat{l}_k}$$

Logits are divided by temperature $T = 0.8$ before argmax selection.

A temperature less than 1 sharpens the distribution, leading to more confident and deterministic outputs.

2

Repetition Penalty

$$\hat{l}_j \leftarrow l_j / r \text{ if } l_j > 0 \text{ and } j \in \text{generated}, \hat{l}_j \leftarrow l_j \times r \text{ if } l_j \leq 0 \text{ and } j \in \text{generated}$$

A multiplicative penalty (with $r = 1.2$) is applied to logits of already-generated tokens, pushing the model away from repetition. Positive logits are reduced, and negative logits are made more negative.

3

N-Gram Blocking

$$\hat{l}_j = -\infty \text{ if } (y_{t-n+1}, \dots, y_{t-1}, j) \in \text{seen n-grams}$$

To prevent exact phrase repetition, any token that would complete a trigram ($n = 3$) already seen in the output is hard-blocked by setting its logit to $-\infty$.

4

Greedy Token Selection

$$y_t = \operatorname{argmax}_j \hat{l}_j$$

The token with the highest modified logit is selected at each step. Generation stops when the End-of-Sequence (EOS) token (ID 3) is generated, or the maximum length of 512 tokens is reached.

Beam Search Decoding



Beam Score Maintenance

$$S_b^{(t)} = \sum_{\tau=0}^t \log p(y_\tau^{(b)} | y_{<\tau}^{(b)}, X)$$

Each beam maintains a cumulative log-probability score.

$$\operatorname{argtop}_B \{S_b^{(t-1)} + \log p_b(j)\}_{(b \in [B], j \in [V])}$$

At each step, all candidate expansions are scored, and the top B hypotheses are retained.



Length Penalty

$$S_{\text{final}}(b) = S_b / |y_b|^\alpha, \alpha = 1.0$$

This penalty normalizes completed beams, preventing beam search from favoring shorter sequences. With $\alpha = 1.0$, each token contributes equally to the score.



Final Selection

$$y^* = \operatorname{argmax}_b S_{\text{final}}(b)$$

The completed beam with the highest length-normalized score is chosen as the final output. Beams reaching max_length without EOS are also considered.



Comparison with Greedy

Beam search explores multiple hypotheses in parallel, potentially finding better solutions than greedy decoding, which can get stuck in local optima. This comes at the cost of increased computation, but typically yields a 2-3 BLEU point improvement in translation tasks.

Model Architecture Summary

Parameter Count Breakdown

Component	Dimensions	Parameters
Shared Embedding W_E	100000 × 768	76.8M
Positional Encoding	Fixed (no params)	0
Encoder × 6 layers		
— Self-Attention (W_Q, W_K, W_V, W_O)	4 × 768²	2.36M per layer
— FFN (W_1, W_2)	768 × 3072 × 2	4.72M per layer
— LayerNorms (×2)	2 × 768	3K per layer
Decoder × 6 layers		
— Masked Self-Attn	4 × 768²	2.36M per layer
— Cross-Attention	4 × 768²	2.36M per layer
— FFN	768 × 3072 × 2	4.72M per layer
— LayerNorms (×3)	3 × 768	4.5K per layer
Output Projection	Tied to W_E	0 (shared)
Total		~250M

Key Hyperparameters

d_model 768	H (attention heads) 12	d_k / d_v 64	d_ff 3072
N_enc / N_dec 6	max sequence length 512	dropout p 0.1	vocab size V 100,000
weight tying Yes	activation GELU	normalization Pre-LayerNorm	init Xavier Uniform

End-to-End Forward Pass

Training Forward Pass:

x_src (token IDs) $\rightarrow E \cdot \sqrt{d}$ (scaled embeddings) $\rightarrow x_emb \rightarrow + PE$
(positional encoding) $\rightarrow x_O$ (encoder input) $\rightarrow \times 6$ Enc layers $\rightarrow enc$
(context) $\rightarrow cross-attn$ (decoder) $\rightarrow h_dec$ (decoder output) $\rightarrow W_E^T$
(output projection) $\rightarrow logits (R^V) \rightarrow CE\ loss \rightarrow L$

Inference Forward Pass (Greedy):

$logits (R^V) \rightarrow / T$ (temperature scaling) $\rightarrow$ repetition penalty $\rightarrow$ n-gram
blocking $\rightarrow$ argmax or beam search $\rightarrow y_t$ (next token)

• Key Points:

- Training uses teacher forcing: decoder receives correct prefix at every step
- Inference is auto-regressive: each token generated sequentially based on previous tokens
- Temperature $T=0.8$ sharpens distribution for more confident predictions
- Repetition penalty $r=1.2$ prevents degenerate loops
- N-gram blocking prevents exact phrase repetition
- Beam search ($B=5$) explores multiple hypotheses for better global solutions
- Generation stops at EOS token (ID 3) or $max_length=512$

This complete pipeline enables the model to translate from English to any of 6 Indian languages (or between them) with high quality and controlled output characteristics.

System Architecture: Frontend & Backend

React Native Frontend (Finterest)

- Built with Expo SDK 52 + TypeScript
- Three tabs: ChatbotScreen, NewsScreen, ProfileScreen
- ChatbotScreen: maintains messages array with role, text, explanation, confidence score
- i18n: react-i18next supports 7 languages (EN, HI, TA, TE, MR, KN, ML)
- Language detected and sent to backend in every request

FastAPI Backend

- Uvicorn server on port 8000
- Axios API service layer with JWT interceptors
- SmartAdapter pattern: Neo4j primary, MockAdapter fallback with auto-reconnect daemon
- CORS enabled for Expo Go development
- Timeout: 30s (Gemini + Neo4j latency budget)

API Endpoints & Request-Response Contract

FastAPI Endpoints

Endpoint	Method	Purpose
/chat	POST	Full multi-hop QA pipeline
/resolve	POST	Entity mention → KG candidate lookup
/translate	POST	EN ↔ Indic text translation
/financial-news	GET	Live NewsAPI feed with 5-min cache
/health	GET	Neo4j connectivity status

/chat Request-Response

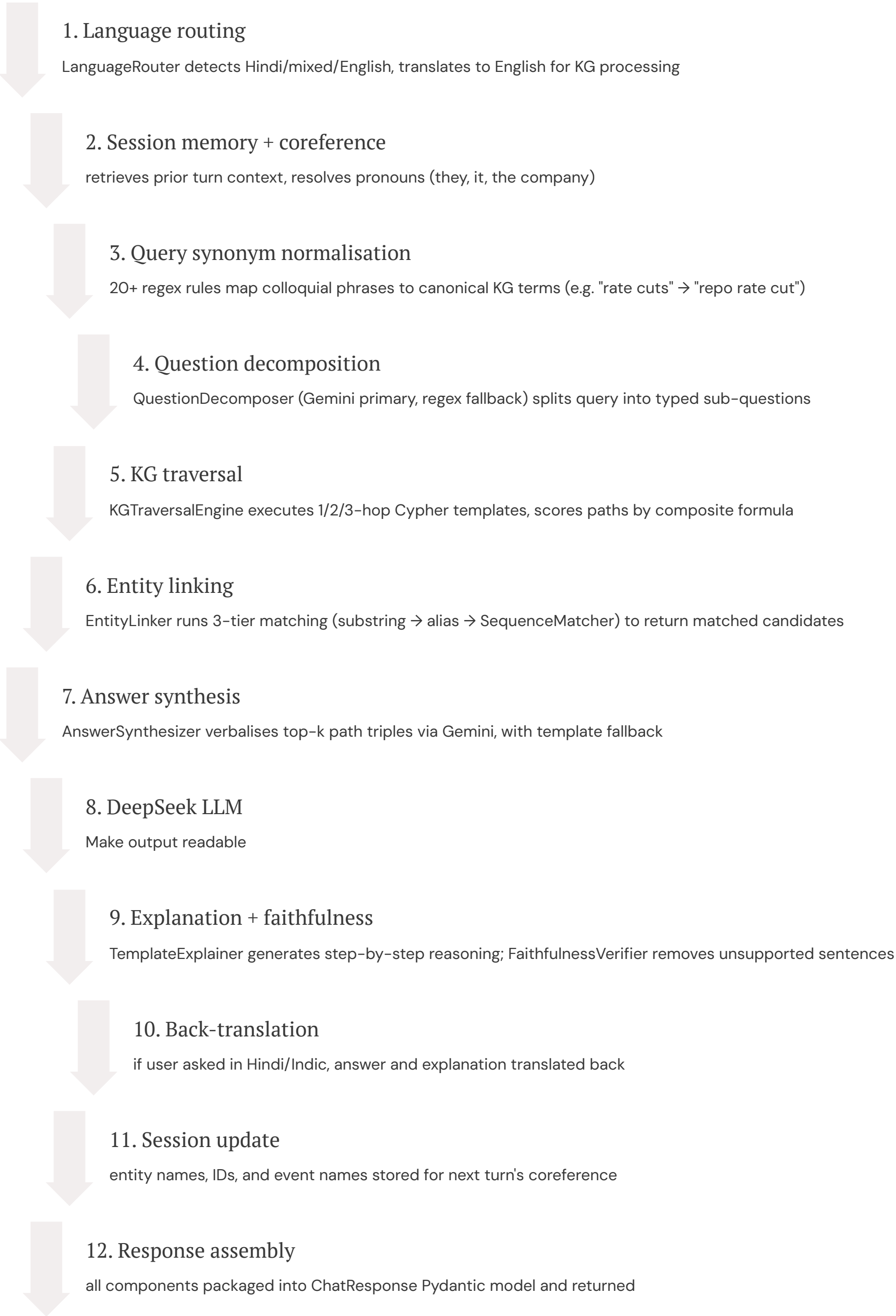
Request body:

- session_id: UUID string (persists conversation context)
- message: user's raw question (any language)
- lang: "en" | "hi" | "auto"
- debug: bool (returns decomposition + timing if true)

Response body:

- answer: synthesized natural language answer
- answer_lang: detected language
- entities: [{id, label, name, score}] — matched KG nodes
- reasoning_paths: [{path_id, hops, triples, score}] — traversal evidence
- explanation_steps: [string] — verified step-by-step reasoning
- citations: [{triple_index, source_uri}] — source grounding
- confidence: float in [0,1]
- warnings: [string] — degradation notices
- debug: {decomposition, query_plan, execution_stats, lang_detection}

12-Step /chat Pipeline



News Feed & Caching Strategy

NewsAPI Integration:

- Endpoint: `GET /financial-news`
- Source: NewsAPI filtered for Indian financial headlines (BSE/NSE/RBI/SEBI domains)
- Rate limit: Free tier allows 100 requests/day
- Cache strategy: Results cached for 300 seconds (5 minutes) in module-level tuple (`fetch_at`, `articles`)
- Fresh cache: returned directly without hitting NewsAPI
- Fallback: If `NEWSAPI_KEY` not set or call fails, returns 5 hardcoded stub articles
- Result: Frontend never sees empty news screen

This ensures reliable news feed even with API quota constraints.

Where the System Falls Short

Despite its advanced architecture, FinTraceQA currently faces several limitations that impact its real-world utility and generalizability.



Limited Data Coverage

The Knowledge Graph is restricted to NIFTY-50 companies and events up to early 2024. Questions about broader markets, mid-cap companies, or newer events fall back to generic responses.



Linking Gaps

Entity linking falters on informal abbreviations, regional transliterations, or companies not in the KG. The relation linker's 70 fixed patterns don't generalize to entirely novel predicate phrasings.



Translation Inaccuracies

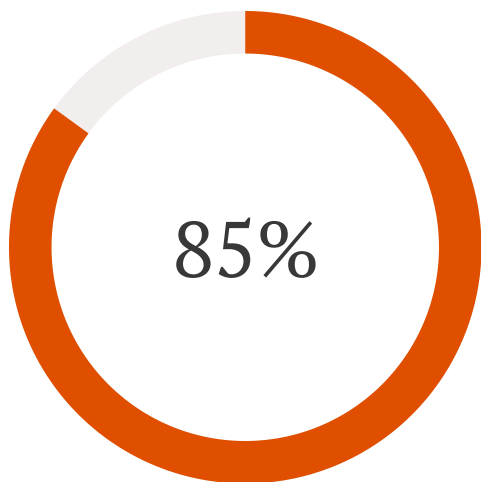
The general-domain translation model struggles with financial terminology, leading to mistranslations that can cause complete entity linking failures and empty traversal results.



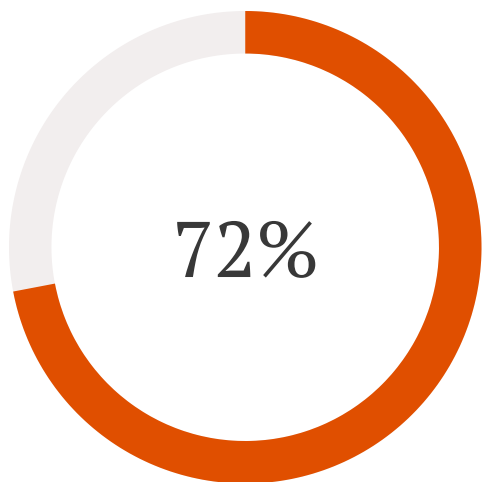
Question Type Restrictions

The system does not support comparative questions (e.g., "Which has a higher P/E?") or yes/no questions, as the traversal engine only produces entity or span answers.

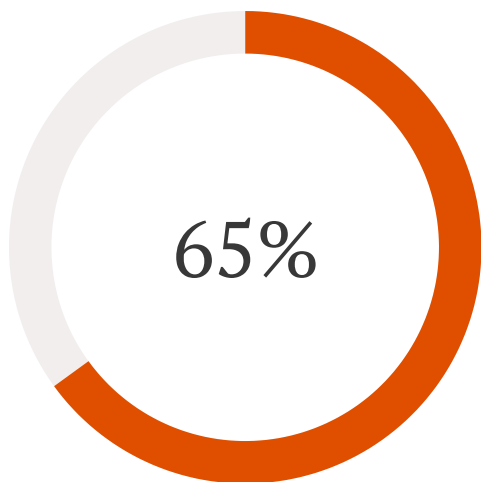
Results, Limitations, and Future Work



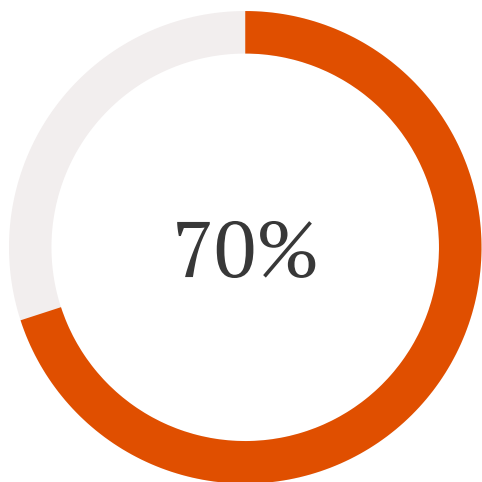
Single-Hop Accuracy
One entity lookup and edge traversal



Multi-Hop Chain
Following chain of relationships



Multi-Hop Intersection
Two simultaneously satisfied paths



Temporal Questions
With time constraints

Current Limitations

- KG covers only NIFTY-50 companies with news corpus ending early 2024
- Translation model trained on general-domain Samanantar, not financial text
- Entity linking fails on informal abbreviations and regional transliterations
- Token-level F1 metric not implemented
- Comparison and yes/no questions not handled

Future Improvements

- Implement token-level F1 evaluation following HotpotQA methodology
- Financial-domain fine-tuning of translation model on Indian financial news corpus
- Expand KG to mid-cap indices, mutual funds, recent news
- Add dense vector retrieval fallback for entities in text but not graph
- Implement comparison question handler with scalar attribute retrieval

📖 **Key References:** Vaswani et al. "Attention Is All You Need" (NeurIPS 2017); Yang et al. "HotpotQA" (EMNLP 2018); Ramesh et al. "Samanantar" (TACL 2022); Kudo "Subword Regularization" (ACL 2018)